



Software de Gestión Administración (DAS) Documento Arquitectura de Software Versión 2.0

**Grupo 1:
Pamela Acuña
Gabriel Martínez
Daniel Castro**



Opacidad Financiera

La dependencia de procesos manuales genera falta de transparencia y desconfianza entre residentes y administración.



Fallas Operativas

Ineficiencia en cobros y retrasos constantes en mantenciones por falta de automatización y trazabilidad.



Limitación Transaccional

Carencia de métodos de pago en línea, forzando transferencias manuales propensas a errores humanos de conciliación.

Implementar una plataforma configurable que optimice la eficiencia administrativa y financiera mediante

- **Motor de Prorrateo:** Cálculo automático por m².
- **Consolidación Híbrida:** Pagos online (Webpay) y manuales.
- **Transparencia:** Estado de cuenta en tiempo real para residentes.
- **Trazabilidad:** Auditoría completa de cada movimiento de caja.



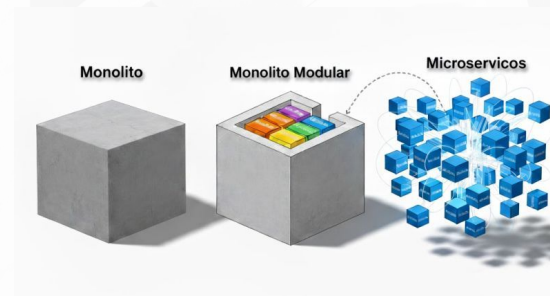
Ciclo de Vida del Proyecto

Fases aplicadas según normativa:

- **Análisis:** Levantamiento de 27 RF y 18 RNF.
- **Diseño:** Definición de estilo modular y patrones UML.
- **Implementación:** Desarrollo en Spring Boot con arquitectura en capas.
- **Pruebas:** Validación de integridad transaccional.

Estilo Adoptado

Arquitectura Monolítica Modular en Capas, que separa responsabilidades y facilita el mantenimiento sin la sobreingeniería de microservicios, integrando APIs externas mediante capas de infraestructura.





MVC

Separación de Modelo (Datos), Vista (Frontend SPA) y Controlador (Gestión de peticiones REST).



Repository & Service

Desacoplamiento total entre la lógica de negocio pura y el almacenamiento físico de datos.

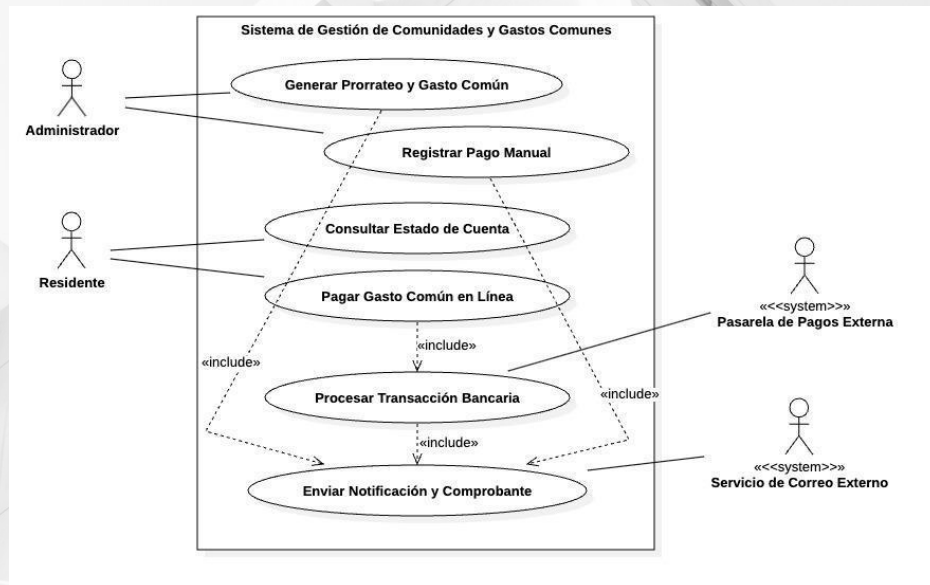


DTO

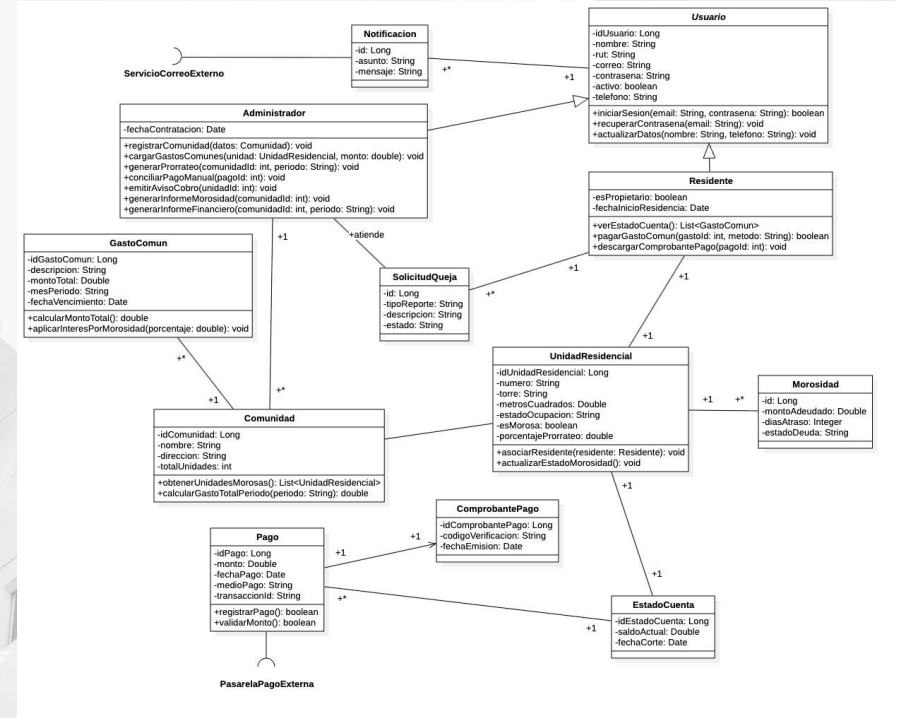
Transferencia segura de datos entre capas, ocultando la estructura interna del modelo de dominio.

Describe el comportamiento desde el punto de vista del usuario final.

- **Administrador:** Gestiona comunidades, prorrates y conciliación manual.
- **Residente:** Consulta estados y realiza pagos online.
- **Sistemas Externos:** Webpay (Pagos) y SendGrid (SMTP).

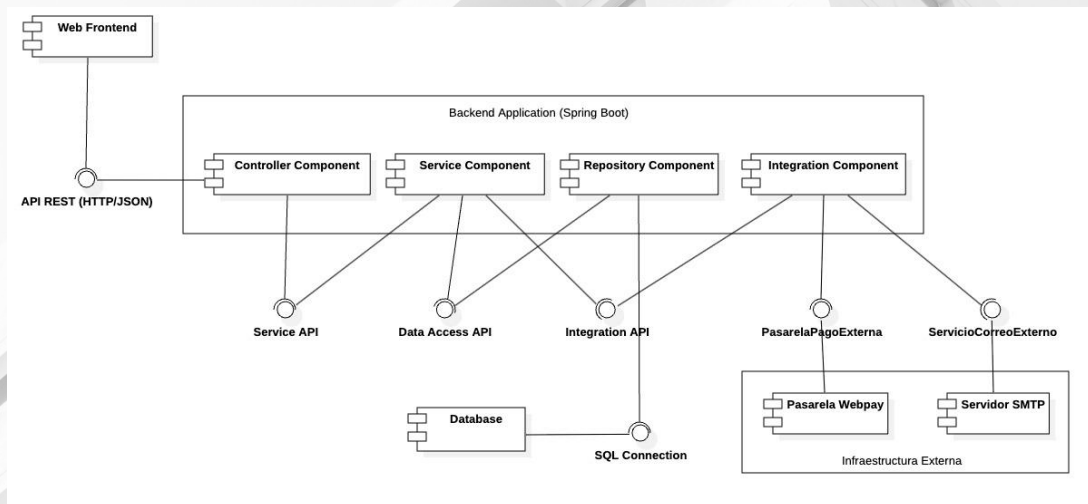


- **Herencia de Usuario:** Generalización para Administrador y Residente.
- **Composición:** Comunidad compuesta por múltiples Unidades Residenciales.
- **Entidades Transaccionales:** GastoComún vinculado a Pagos validados.



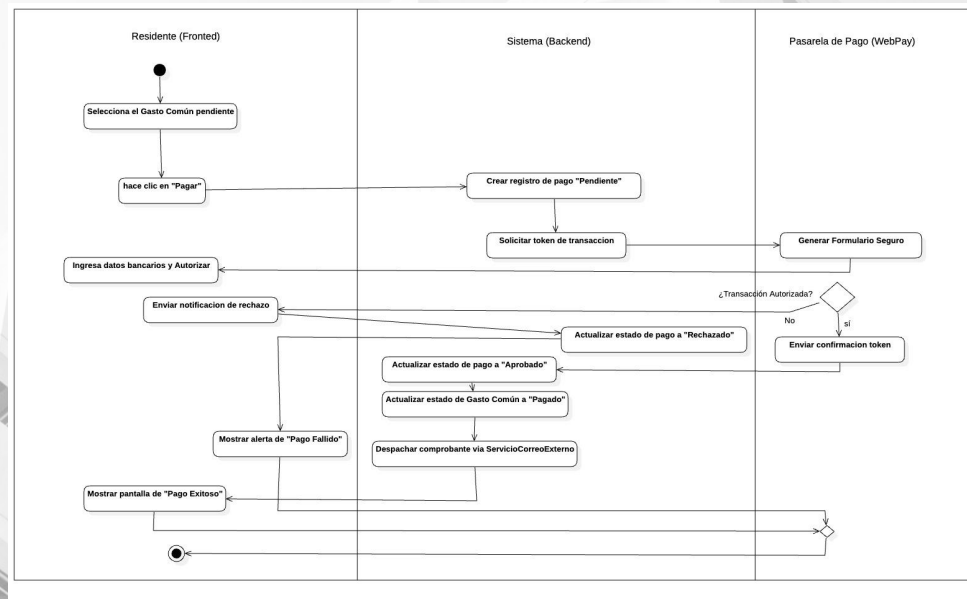
Organización física del código en paquetes modulares:

- **Controller:** Endpoints de la API REST.
- **Service:** Lógica de negocio y orquestación.
- **Integration:** Adaptadores para APIs externas (Webpay/Email)
- **Repository:** Abstracción de persistencia SQL



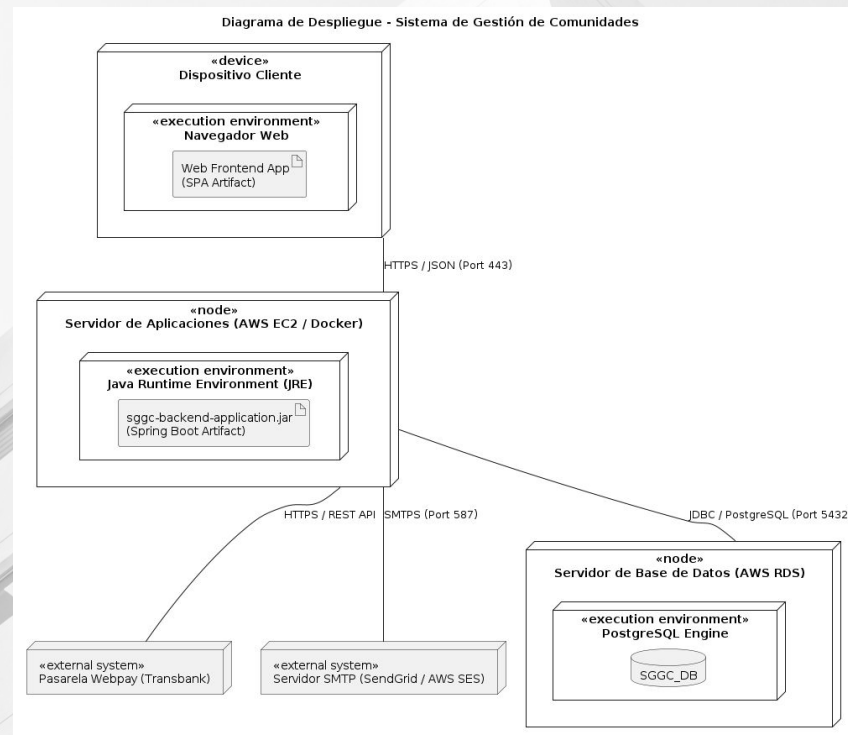
Comportamiento dinámico durante la recaudación:

- **Flujo Sincrónico:** Validación con la Pasarela de Pago Externa
- **Actualización en Cadena:** Cambio de estado de deuda y baja de saldos.
- **Notificación:** Gatillo de comprobante digital asíncrono.



Topología de red y distribución en la nube:

- **Servidor Aplicaciones:** AWS EC2 bajo Docker / JRE.
- **Persistencia:** AWS RDS con PostgreSQL 16+.
- **Protocolos:** HTTPS (443) y SMTPS (587).
- **Frontend:** Artefacto SPA interpretado en el cliente



ATRIBUTO	ESTÁNDAR / MÉTRICA	JUSTIFICACIÓN
Usabilidad	Heurísticas de Nielsen	Interfaces intuitivas para residentes con distintos niveles digitales.
Rendimiento	T. Respuesta < 3s	Garantiza eficiencia en la consulta de estados de cuenta masivos.
Seguridad	Encriptación HTTPS / JWT	Protección de datos personales y trazabilidad de cambios manuales.
Accesibilidad	Normativa WCAG	Inclusión digital para todos los miembros de la comunidad

